Can Deterministic Network Verification Reduce Undetected Faults in LLM-Generated Configurations?

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

***Abstract*—We preregistered and executed a paired confirmatory study (study id c1-gnr-vv-2026-0001) to measure whether inserting a deterministic network verifier between an LLM intent→configuration generator and an emulated execution reduces the proportion of configurations that would execute with operational faults. Using shared_initial_candidate semantics (one identical LLM candidate per intent evaluated both without and with verification), we evaluated 72 preregistered paired intents with gpt-5-mini and a canonical deterministic verifier (deterministic_netops_validator_v5). Execution completed with 144 episodes (72 pairs) and 72 model calls. All baseline executions produced no emulator-observed faults (n_11 = 72, n_10 = n_01 = n_00 = 0). The paired difference in undetected-fault proportion is 0.0 (paired bootstrap 95% CI [0.0, 0.0], exact McNemar two-sided p = 1.0; bootstrap seed = 1729, resamples = 10000). Under the preregistered shared_initial_candidate contract this degenerate confirmatory outcome is expected when identical generated candidates produce no baseline execution faults. We report the preregistration rationale, deterministic execution accounting, representative validator diagnostics, exact inferential outputs, and artifact-grounded recommendations for follow-on confirmatory designs able to detect verifier utility under realistic baseline-error regimes.**

I. Introduction

Generative large language models (LLMs) are increasingly proposed to translate high-level operator intents into device-level network configurations. This intent→configuration automation can speed change pipelines but risks silently violating reachability, access-control, ordering, or workflow invariants and thereby causing outages if generated text is executed without additional checks. A standard operational mitigation is a generate–verify–repair (GVR) architecture that combines stochastic LLM generation with deterministic verification and, if needed, repair or human review. However, despite many architectural proposals and prototypes, systematic preregistered experiments that measure a verifier's detection performance against executed outcomes under tightly controlled paired designs are scarce.

This paper answers a narrowly scoped, preregistered research question: does inserting a deterministic network verifier between an LLM generator and emulator execution materially reduce the proportion of configurations that execute with operational faults when the identical generated candidate is evaluated both without and with verification? That estimand isolates verifier detection from improvements that could arise from re-sampling, prompt-engineering, or repair-enabled iterations. We executed study c1-gnr-vv-2026-0001 using gpt-5-mini and deterministic_netops_validator_v5 across 72 preregistered paired intents under shared_initial_candidate semantics. The run completed with no technical failures, produced a degenerate confirmatory outcome (no baseline emulator faults), and yields a paired difference of 0.0 (95% CI [0.0,0.0], exact McNemar p = 1.0). Rather than treating this as a null failure, we analyze why it occurred given the preregistered contract and archived artifacts, and we provide concrete, artifact-grounded guidance for designs that can detect verifier utility when baseline error prevalence is non-negligible.

II. Related Work

Work on LLM-driven NetOps spans intent→configuration translation, multi-agent/tool-augmented orchestration, and architectural proposals for combining stochastic generation with deterministic checks. Prior empirical studies demonstrate that modern LLMs can produce device-level configurations from high-level intent in constrained vendor-dialect settings and curated evaluation suites; these results show promise but are typically scoped to limited task families and curated datasets rather than broad production traces [1]. That line of work motivates leveraging LLMs to reduce operator effort while highlighting the need for safety controls when configuration semantics and ordering matter.

Complementing single-agent generation work, multi-agent and tool-augmented frameworks have been proposed that explicitly integrate verifiers, tool calls, or specialized modules into LLM-driven NetOps pipelines to reduce regressions and coordinate multi-step changes [2]. These systems illustrate practical architectures for inserting deterministic checks or domain services into agentic workflows, and their evaluations commonly rely on emulators, curated scenarios, or prototype deployments. Such prototypes suggest feasible integration points for verifiers but do not, in the supplied records, provide preregistered paired measurements of verifier detection performance against executed faults.

A related and more general strand of work articulates and evaluates generate–verify–repair (GVR) patterns outside NetOps—particularly in code- and software-repair domains—where stochastic generators produce candidates that deterministic checkers validate and, when necessary, trigger repair iterations [3]. These studies document how deterministic validation can convert nondeterministic generation into a correctness-convergent workflow; they also show that the empirical util-

ity of verification depends strongly on the baseline error rate of generator outputs and on whether the architecture permits verifier-triggered repairs or resampling. Translating these insights into NetOps requires coupling LLM outputs to domain-appropriate verifiers and measuring detection relative to executed outcomes.

Canonical deterministic network-verification techniques—exemplified by header-space analysis and related formal reachability/policy analyzers—provide precise, semantics-aware invariants for reachability and ACL-style policy checks and are natural candidates for the 'verify' stage in GVR architectures [4]. Such verifiers can deterministically flag violations of specified invariants or workflow constraints expressed over packet headers, interface states, and rule orderings. However, the literature lacks, among the supplied records, preregistered paired experiments that measure how often these canonical verifiers would have detected faults that would otherwise manifest when LLM-generated configurations are executed in an emulator or live device.

Taken together, the verified literature establishes three pertinent points: (i) LLMs can produce plausible device-level configs from intent in constrained settings [1]; (ii) system proposals and prototypes show how verifiers and tools can be integrated into multi-agent NetOps workflows [2]; and (iii) GVR patterns can yield empirical correctness improvements in other domains when baseline generator errors are non-negligible and when verifier-triggered repairs or resampling are allowed [3]. What remains underexplored in the supplied evidence is a preregistered, paired measurement that isolates verifier detection on identical candidates—i.e., does a canonical deterministic verifier reduce the probability of an execution fault for the same LLM-generated candidate when the candidate is executed with and without prior verification? Our study directly targets this measurement gap by adopting a shared_initial_candidate paired design and reporting exact execution-accounting artifacts to make that detection question reproducible and auditable.

III. Methodology

Preregistration and primary estimand. We preregistered and froze study c1-gnr-vv-2026-0001 before observing outcomes. The primary estimand is the paired reduction in undetected operational-fault proportion (paired_success_rate_difference_guarded_minus_baseline).
The confirmatory hypothesis (H1) targeted an absolute paired reduction of 0.20 (two-sided alpha = 0.05, power >= 0.80); a sample-size heuristic yielded ≈63 pairs and we preregistered N = 72 pairs (24 per topology-difficulty stratum). The preregistration and analysis plans, including sensitivity analyses and exclusion rules, are part of the archived preregistration record.

Shared_initial_candidate semantics and experimental contract. The execution_contract enforced shared_initial_candidate semantics: for each intent we performed exactly one initial LLM generation and evaluated that identical candidate in two paired conditions. Baseline condition: execute the shared candidate in an isolated emulator and record whether emulator-observed operational faults occur. Guarded condition: run the canonical deterministic verifier on the same candidate; if the verifier flags violations the guarded outcome is recorded as prevented (no executed fault); if the verifier does not flag, execute the identical candidate in the emulator and record the result. By construction, any observed paired difference can only arise from the verifier preventing execution of a candidate that would otherwise have failed at emulation; this isolates detector utility from generator-side resampling or repair effects.

Model, adapter, and verifier configuration (artifact-grounded). The declared model is gpt-5-mini (execution/model_configuration.json), requested via the hosted_netops_gvr_v1 adapter. Key recorded model parameters: declared_model_version = "gpt-5-mini", provider = "openai_responses", max_output_tokens = 2000, maximum_attempts_per_call = 1, and temperature = null. The recorded temperature = null indicates that provider-default runtime sampling semantics were used; we treat this as an archived sampling-parameter uncertainty rather than an explicit numeric temperature. The canonical verifier is deterministic_netops_validator_v5 (validator_version = "5.0") configured to run the full_intent_constraint_validation rule set. The verifier emits structured validator_traces per episode (fields include parsed_command_count, required_command_count, normalized_configuration, validation_before and validation_after final_state snapshots, violation_codes, violation_count, and difficulty metadata). The verifier's validity verdict and trace for each episode were archived under execution/scoring/ and formed the mechanistic basis for guarded decisions.

Task-bank construction, contamination controls, and stratification. Confirmatory tasks were generated deterministically by deterministic_netops_task_generator_v5. Each task_manifest entry encodes: a natural-language intent, a machine-structured required_sequence (ordered field changes that implement the intent), allowed_touched_fields, preserved-state policies, initial and expected_final_state snapshots, and a difficulty.level tag (medium/hard/very_hard). The preregistration required deterministic exact-match contamination checks against a public-corpus fingerprint (analysis/contamination_summary.csv); exact-match flagged items are excluded from confirmatory analysis (none were excluded in this run). The confirmatory sample followed the preregistered stratification (24 pairs per difficulty stratum) to allow descriptive subgroup inspection.

Execution protocol, retries, and deterministic accounting. The missingness_plan permitted up to two retries (three attempts total) for transient hosted-API errors on generation calls; all retries and provider events were logged. The run started at 2026-08-31T06:56:23.065707Z and completed at 2026-08-31T07:06:28.665518Z (execution manifest). The execution manifest records planned_episode_count = 144, completed_episode_count = 144, failed_episode_count = 0, and model_calls_used = 72 (one shared generation per

pair). Provider-call audit (deterministic_reconciliation.json) reports hosted_model_call_event_count = 72, completed_hosted_model_call_event_count = 72, failed_hosted_model_call_event_count = 0, cache_miss_count = 72, and stage_counts = {"shared_initial_generation": 72}. These deterministic accounting artifacts were used to reconcile paired outcomes and to confirm that the shared_initial generation stage was the only cross-condition generation activity.

Scoring, validation rules, and per-episode traces. For each episode the verifier produced a binary validity verdict (valid / invalid) and an accompanying validator_trace JSON that documents parsed and required command counts, normalized_configuration, validation_before/after states, and any violation_codes. The scoring rule deployed was full_intent_constraint_validation (validator enforces required_sequence order, allowed_touched_fields, and preserve_unspecified_state constraints encoded in the task manifest). Per-episode scoring artifacts (execution/scoring/task-*.json) therefore permit audit of why a candidate was accepted or flagged. In the recorded run no verifier flagging occurred for confirmatory pairs (violation_count = 0 across representative episodes), and no automated repair was applied (repair_applied = false in validator_trace fields).

Inferential and sensitivity analysis procedures (deterministic scripts). The primary analysis used the registered paired_binary_analysis_v1 executor to construct the 2×2 contingency table (guarded $\times$ baseline) using binary indicators where an undetected executed fault is 1 and success/no-fault is 0. The prespecified exact inferential test is an exact McNemar two-sided test when discordant pairs exist. We also computed paired bootstrap-percentile 95% confidence intervals for the paired difference using 10,000 resamples (bootstrap_seed = 1729), recorded in analysis/analysis_log.jsonl. Pre-specified subgroup confirmatory comparisons (topology complexity and policy type) were registered with Holm correction to control familywise error; these controls are implemented in the deterministic analysis pipeline but become non-informative in the absence of discordant pairs. Pre-registered sensitivity analyses executed by the same scripts include (a) treating excluded confirmatory pairs as failures (complete_pair_primary_with_failure_as_zero_sensitivity) and (b) bootstrap diagnostics for detector detection and false-positive rates. Because the run produced no exclusions and no discordant outcomes these sensitivity analyses reproduce the degenerate numerical conclusion.

Design-level notes and construct tradeoffs (artifact-linked). The shared_initial_candidate semantics intentionally isolates verifier detection from generator-side sampling; this preserves a clean causal interpretation of verifier-prevention utility but prevents measurements that rely on verifier-triggered resampling or repair. The archived execution/model_configuration.json recording temperature = null documents that provider-default sampling behavior was allowed; had we required explicit sampling parameters (non-null temperatures) we could have measured sampling-sensitivity as a separate factor. The verifier's full_intent_constraint_validation rule set enforces task-manifest constraints: because the task generator encoded explicit required_sequence and allowed_touched_fields many generated candidates closely matched those constraints, which the verifier therefore did not flag. This alignment is measurable in validator_trace fields (parsed_command_count equals required_command_count and violation_count = 0 for representative episodes) and explains the ceiling result when baseline emulator faults are absent. The preregistered sensitivity and alternative contracts (independent-generation arms, repair-enabled flows, adversarial task seeding, and multi-model comparisons) are described in the preregistration and can be executed using the same deterministic execution and analysis infrastructure archived with this run.

IV. Results

Execution and manifest summary. The autonomous pipeline completed without technical failures and with full deterministic accounting. Key run-level figures from the archived manifests are: planned_episode_count = 144; completed_episode_count = 144; failed_episode_count = 0; model_calls_used = 72; artifact_count = 510. The run-level timestamps (started_at_utc = 2026-08-31T06:56:23.065707+00:00; completed_at_utc = 2026-08-31T07:06:28.665518+00:00) indicate a wall-clock duration of $\approx$10.1 minutes. analysis/condition_summary.csv (archived) records baseline successes = 72, baseline failures = 0 (success_rate = 1.0) and guarded successes = 72, guarded failures = 0 (success_rate = 1.0).

Paired contingency, inferential outputs, and bootstrap diagnostics. Aggregating across the 72 confirmatory pairs produced the exact paired contingency counts n_11 = 72, n_10 = 0, n_01 = 0, n_00 = 0 (guarded $\times$ baseline). The primary estimand paired_success_rate_difference_guarded_minus_baseline = 0.0. The preregistered exact McNemar two-sided test produces p = 1.0 given the absence of discordant pairs. The paired bootstrap-percentile 95% confidence interval for the paired difference (bootstrap_seed = 1729; bootstrap_resamples = 10000; analysis/analysis_log.jsonl) is [0.0, 0.0], reflecting the degenerate sampling distribution when all paired outcomes are concordant. analysis/paired_contingency_table.csv and analysis/condition_summary.csv contain these tabulations and are archived.

Per-episode validator diagnostics and representative exemplars. We inspected archived per-episode scoring JSONs under execution/scoring/ and representative task-manifest entries under manuscript evidence. Across the sampled exemplars the verifier produced no violation flags. Representative excerpts from archived scoring artifacts: - pair-000001 (task-000001): difficulty.level = "medium"; parsed_command_count = 4; required_command_count = 4; validator_version = "5.0"; violation_count = 0; normalized_configuration equals the shared candidate (execution/scoring/task-000001-*.json).

- pair-000002 (task-000002): difficulty.level = "hard"; parsed_command_count = 2; required_command_count = 2; violation_count = 0. - pair-000003 (task-000003): difficulty.level = "hard"; parsed_command_count = 6; required_command_count = 6; violation_count = 0. For additional archived representative tasks the task manifests encode required_command_count and difficulty tags (examples: task-000004 required_command_count = 4, difficulty.level = "very_hard"; task-000005 required_command_count = 3, difficulty.level = "very_hard"; task-000006 required_command_count = 6, difficulty.level = "very_hard"). In all archived scoring JSONs for confirmatory pairs the verifier_traces report violation_count = 0 and validation_after.valid = true, demonstrating that the verifier processed each shared candidate and did not flag the candidate as violating the manifest-encoded constraints.

Contamination, missingness, and sensitivity analyses. analysis/contamination_summary.csv records zero exact-match contamination flags for confirmatory pairs. analysis/missingness_summary.csv records complete_pairs = 72 and zeros for failed or unscorable episodes. The preregistered sensitivity analyses (including the conservative treatment that would count excluded pairs as failures and bootstrap diagnostics) were executed by the registered analysis executor; because there were no exclusions and no discordant pairs these sensitivity analyses reproduce the degenerate numeric conclusion (paired difference = 0.0). All analysis artifacts (analysis/paired_contingency_table.csv, analysis/condition_summary.csv, analysis/analysis_log.jsonl) are archived and reference the bootstrap seed and resample count used.

Deterministic reconciliation and provider-call audit. The deterministic_reconciliation.json confirms accounting consistency: completed_episode_count = 144, complete_pair_count = 72, baseline_success_count = 72, guarded_success_count = 72, n_11 = 72, n_10 = 0, n_01 = 0, n_00 = 0, and all_deterministic_consistency_checks_passed = true. Provider-call audit fields show hosted_model_call_event_count = 72, completed_hosted_model_call_event_count = 72, failed_hosted_model_call_event_count = 0, cache_miss_count = 72, and stage_counts = {shared_initial_generation: 72}, consistent with the preregistered shared_initial_candidate semantics and the execution manifest.

Interpretation of the degenerate outcome (evidence-grounded). The complete absence of baseline emulator faults in the archived confirmatory corpus is the proximate cause of the degenerate paired result. The archived artifacts together document measurable contributors: (1) the deterministic task generator produced constrained intents with manifest-encoded required_sequence and allowed_touched_fields that align the generation objective tightly with verifier checks; (2) gpt-5-mini (declared in execution/model_configuration.json) produced configurations that matched those manifest constraints across the sampled tasks; and (3) the deterministic_netops_validator_v5 rule set (full_intent_constraint_validation) enforces the same manifest-encoded sequence and field constraints, producing concordant validity judgments. These archived, machine-verifiable facts explain why the verifier could not reduce executed faults under the shared_initial_candidate contract: there were no executed faults to detect.

V. Discussion

Confirmatory interpretation and boundary conditions. The preregistered paired design with shared_initial_candidate semantics validly measures verifier detection on identical candidates; because the identical generated candidates produced zero emulator faults the verified paired outcome is a ceiling/null result: the verifier could not reduce executed faults that did not occur. The numeric outputs (n_11 = 72; paired difference = 0.0; McNemar p = 1.0; bootstrap CI [0.0,0.0]) are direct consequences of the preregistered contract and observed data and therefore correctly reported as a degenerate confirmatory result.

Artifact-grounded causes of the ceiling outcome. Archived artifacts allow us to identify measurable factors that explain the ceiling outcome: (1) the deterministic task generator produced constrained, high-clarity intents with explicit required_sequence fields and allowed_touched_fields encoded in each task manifest, increasing the likelihood of unambiguous correct candidates; (2) gpt-5-mini (declared_model_version in execution/model_configuration.json) generated sequences that matched those structured constraints across the synthetic corpus; and (3) the verifier rule set (full_intent_constraint_validation in deterministic_netops_validator_v5) aligns closely with the required_sequence constraints present in the task manifests, making verification and generation objectives strongly congruent. These archived, measurable factors together account for the absence of baseline emulator faults.

Design implications: when verification benefit is measurable. To empirically measure practical verifier benefits, confirmatory contracts must present non-negligible baseline error prevalence or allow verifier-mediated candidate changes. Artifact-grounded, preregisterable alternatives include: (A) independent-generation arms—allow separate initial generations per condition so verifier-triggered repairs or alternative sampling may change executed candidate distributions; (B) repair-enabled flows—permit a deterministic repair call when the verifier flags violations and execute repaired candidates to measure end-to-end improvement; (C) adversarial/fault-injection task banks—seed tasks with ambiguous intents, ordering subtleties, or vendor-edge cases to raise baseline error prevalence; (D) multi-model and sampling sweeps—include multiple model versions and explicit sampling parameter settings (non-null temperatures) to quantify model- and sampling-sensitivity. All these options are compatible with the archived execution and analysis infrastructure and can be preregistered using the same evidence-recording conventions.

Threats to validity revisited (artifact-supported). Internal validity: by design the shared_initial_candidate semantics isolates detection but prevents verifier-mediated candidate resam-

pling from influencing outcomes; this tradeoff explains why a degenerate result may occur when baseline error prevalence is low. Construct validity: emulator-observed faults are a conservative proxy for operational harm but do not capture traffic-dependent timing, vendor-specific hardware idiosyncrasies, or live-traffic emergent failures. External validity: experiments used a single declared model (gpt-5-mini), a synthetic task bank, and one deterministic verifier; extrapolation to other models, vendor dialects, or production hardware is limited. Conclusion validity: because baseline error prevalence was zero in this corpus the confirmatory design had no power to detect verifier benefit; the preregistered power target (detect absolute paired reduction 0.20) becomes moot under a zero-prevalence baseline.

Operational implications and measured tradeoffs. The confirmed ceiling outcome does not imply verifiers are unnecessary; rather, it shows that under certain tightly constrained synthesis and evaluation conditions an LLM alone produced valid device sequences on the synthetic corpus. In production settings with ambiguous intents, incomplete constraints, or vendor heterogeneity, verifiers remain a mechanistic guard. Measuring verifier costs (false positives that block safe changes) and benefits (true positives preventing faults) requires task banks with non-negligible baseline failure prevalence; the archived artifacts and preregistration provide a reproducible template for such follow-on evaluations.

Reproducibility notes (artifact pointers and deterministic checks). All execution and analysis artifacts are archived in the evidence bundle. Key artifacts referenced in this paper include: execution/raw_results.jsonl, execution/task_manifest.jsonl, execution/model_configuration.json, analysis/paired_contingency_table.csv, analysis/condition_summary.csv, analysis/analysis_log.jsonl, deterministic_reconciliation.json, and per-episode validator traces under execution/scoring/. The analysis bootstrap used seed = 1729 and 10,000 resamples; analysis/analysis_log.jsonl records these values. The run-level timing, provider-call audit, and artifact counts are recorded in execution/ and analysis/ manifests to enable deterministic reconciliation of the paired accounting. The model configuration records temperature = null (execution/model_configuration.json), which we report as provider-default sampling semantics (a residual sampling-parameter uncertainty archived in the evidence).

VI. Limitations

- Confirmatory ceiling/null outcome: the baseline generator produced zero emulator faults on the preregistered task set, preventing direct measurement of verifier detection benefit.
- Shared_initial_candidate semantics: the design measures verifier effect on the same generated candidate only and does not assess independent per-condition generation, multi-sample generation, or repairs unless explicitly permitted by the contract.
- Single-model scope: experiments used one declared model (gpt-5-mini); results do not imply generality across models or versions.
- Synthetic/emulated tasks: task corpus and emulator executions differ from production devices and live traffic; external validity to production deployments is limited.
- Sampling-parameter uncertainty: execution/model_configuration.json shows temperature = null (sampling parameters unset); null indicates provider-default runtime sampling semantics and is a residual uncertainty recorded in the archived configuration.
- Residual contamination risk: deterministic provenance and exact-match checks were applied, but private training-data contamination of the hosted model cannot be fully ruled out and remains residual uncertainty.

VII. Conclusion

We executed a preregistered paired evaluation (c1-gnr-vv-2026-0001) under shared_initial_candidate semantics to test whether a canonical deterministic verifier reduces the proportion of LLM-generated configurations that would execute with operational faults. For the chosen model (gpt-5-mini), verifier (deterministic_netops_validator_v5), and synthetic/emulated task bank, baseline executions produced zero emulator faults and the verifier did not change outcomes (paired difference = 0.0; bootstrap 95% CI [0.0,0.0]; exact McNemar p = 1.0). This artifact-grounded null/ceiling outcome is internally consistent with the preregistered design: when identical generated candidates produce no executed faults a verifier cannot reduce executed faults under the shared_initial_candidate contract. Measuring verifier utility under realistic error regimes requires preregistered contracts that permit independent or multiple generator samples, repair-enabled flows, adversarial task sets, and multi-model comparisons. The archived artifacts (responses, task manifests, validator traces, execution manifest, and analysis logs) provide a reproducible baseline and a template for those follow-on confirmatory designs and power calculations.

Disclosure Statement

Autonomy and artifacts: This study was executed autonomously after an autonomy lock. Human role: a Research Director selected the topic family and approved the immutable initial master prompt prior to the autonomy lock; no human manually edited technical manuscript content after the lock. Models and tools: initial generation used gpt-5-mini (declared_model_version recorded in execution/model_configuration.json) orchestrated via the hosted_netops_gvr_v1 adapter; deterministic_netops_validator_v5 (validator_version = "5.0") served as the canonical verifier; an isolated emulator executed candidate configurations. Preregistration: study c1-gnr-vv-2026-0001 was preregistered and frozen before observing outcomes. Immutable master prompt reference: provenance/master_prompt.txt (SHA-256: 1872df1e1805d2d96940456ca016bd66

5d1d5196add77f5acdf1582bb39b15ba). Key archived artifacts include execution/raw_results.jsonl, execution/task_manifest.jsonl, execution/model_configuration.json, analysis/paired_contingency_table.csv, analysis/condition_summary.csv, deterministic_reconciliation.json, and per-episode validator traces under execution/scoring/. The model configuration records temperature = null (provider-default sampling semantics archived in execution/model_configuration.json). Provider-call audit: hosted_model_call_event_count = 72. No human manual manuscript editing or content insertion occurred after the autonomy lock.

REFERENCES

[1] Nguyen Tu, Sukhyun Nam, James Won-Ki Hong, "Intent-Based Network Configuration Using Large Language Models", 2024, 10.1002/nem.2313

[2] Zhaodong Wang, Samuel Lin, Guanqing Yan, Soudeh Ghorbani, Minlan Yu, Jiawei Zhou, Nathan Hu, Lopa Baruah, Sam Peters, Srikanth Kamath, Jian Yang, Ying Zhang, "Intent-Driven Network Management with Multi-Agent LLMs: The Confucius Framework", 2025, 10.1145/3718958.3750537

[3] Rafael Cadenas, "Convergent Correctness in Stochastic Code Generation: A Generate-Verify-Repair Architecture for Deterministic Validation of Autonomous AI Coding Agents in Regulated Environments", 2026, 10.2139/ssrn.6754899

[4] http://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.300.8659